

INTERNAL DOCUMENTATION

API Automation Dashboard Walkthrough Guide

A complete tour of the Playwright test-report analytics dashboard — how it turns raw automated-test output into pass/fail trends, failure analysis, and tenant comparisons for the QA team.

Prepared for: Manager & Team Walkthrough

Date: September 9, 2026

Scope: All pages, filters, exports and data sources currently in the application

Contents

1 Overview — What This Dashboard Is

2 How Data Gets In — Upload & Parsing Pipeline

3 Navigation & Global Filters

4 Page Walkthrough

4.1 Upload

4.2 Trends (Home Page)

4.3 Failures / Analysis

4.4 Scenarios

4.5 Tenant Comparison

4.6 Single-Report Analysis

4.7 Drill-Down (Test List)

5 Exports — PDF, CSV & Excel

6 Access & Security Notes

7 Glossary of Terms

1. Overview — What This Dashboard Is

The **API Automation Dashboard** is an internal web application that turns raw **Playwright test results** into readable, visual QA reporting. When our automated test suites run (API tests and UI tests), Playwright produces a JUnit-style XML results file. Someone uploads that file to this dashboard, and the app parses it into charts, tables, and drill-down views so anyone — from a QA engineer to a manager — can quickly answer questions like:

- How many tests passed / failed / were skipped in the latest run?
- Is our pass rate trending up or down over the last several runs?
- Which specific tests fail *every* time, and which are flaky (inconsistent)?
- Did a test that used to pass suddenly break (a regression)?
- Does a given API behave differently for different customer tenants?

The tool is purpose-built around **our own test suites** and currently tracks three distinct automated-testing efforts, referred to throughout the app as **"Report Types"** or **pipelines**:

Pipeline (as shown in the UI)	What it covers
Estimation API Tests	General backend/API automated test suites
Estimation AI-API Tests	The Estimation AI product's suite, run against its API layer
Estimation AI-UI Tests	The Estimation AI product's suite, run through a real browser (UI automation)

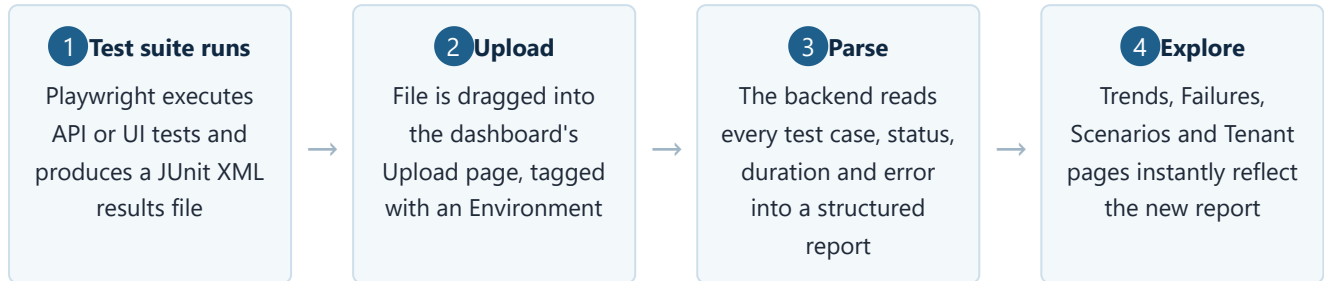
Nearly every page lets you filter by one of these pipelines, or view **"All Tests"** combined.

TECH SNAPSHOT

The app is a React/TypeScript front end backed by a small Node.js/Express service. There is no database — each uploaded XML file is stored as-is on disk and re-read/parsed whenever a page needs it. This keeps the tool lightweight and easy to run, though it means every "cross-run" page (Trends, Failures, Scenarios) works by pulling in every matching report and combining them in the browser.

2. How Data Gets In — Upload & Parsing Pipeline

Every insight in the dashboard traces back to one action: uploading a Playwright XML results file. Here's the end-to-end flow:



What gets extracted from each XML file

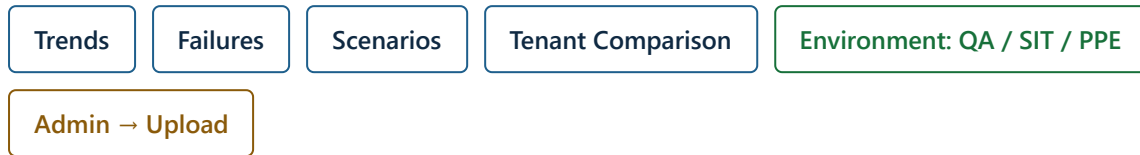
Field	Where it comes from
Pass / Fail / Skipped / Flaky status	Each test case's outcome in the XML, with "flaky" synthesized where applicable
Duration	Time recorded per test and per suite
Error message & stack trace	The <code><failure></code> content of a failed test case
Error category	Auto-classified by pattern-matching the error text into: Assertion, Timeout, Network, Element-not-found, Runtime, or Application
Tenant ID	Parsed from logged lines such as <code>[INFO] TenantId: 4</code> in a test's output, or from the file name as a fallback
UI vs. API classification	Inferred from the test suite's "hostname" — real-browser hostnames mean UI, device-matrix hostnames mean API
Environment (QA / SIT / PPE)	Chosen manually at upload time — not detected from the file content

DUPLICATE PROTECTION

Each report is fingerprinted (a hash of its file contents). Uploading the exact same file twice is blocked with a message linking straight to the existing report — so accidental double-uploads can't skew the trend charts.

3. Navigation & Global Filters

The app uses a single top navigation bar — there is no sidebar. From left to right: the logo/title, four main page links, an Environment selector, and an Admin menu.



Environment selector (global)

A dropdown for **QA**, **SIT**, or **PPE**. Whichever one is selected is remembered in the browser and applied to *every* page — all charts, tables and filters only ever show reports uploaded to that environment. Switching environments briefly shows a loading overlay while the view refreshes.

Report Type / Pipeline selector

Present on the Trends, Failures and Scenarios pages. Lets you narrow the page to one of the three pipelines described in Section 1, or choose "**All Tests**" to see everything combined. The options offered adapt to the selected environment — for example, **PPE** currently only has data for **Estimation AI-UI Tests**, so the other two pipeline options aren't offered there, and the filter automatically resets to "All Tests" if you switch to an environment where your previous pipeline choice doesn't exist.

Admin menu

Deliberately kept separate from the main navigation — its only item is **Upload**, which is where new XML reports are added. Keeping this out of the main nav avoids clutter for people who only need to view results.

4. Page Walkthrough

The following pages cover every screen in the dashboard, in the order a first-time user would naturally encounter them: upload a report, land on the Trends overview, then branch into deeper analysis (Failures, Scenarios, Tenant Comparison), and finally drill into one specific report or test list.

4.1 Upload

/upload — reached via Admin → Upload

The entry point for getting a new test report into the system.

- **Drag-and-drop zone** (or click to browse) accepts a single `.xml` results file, shows the file name/size once selected, and can be removed before submitting.
- **Analyze Report** button uploads the file with a progress bar, tags it with the currently selected Environment, and — on success — jumps straight to the Trends page so the new data is immediately visible.
- If the identical file was already uploaded, an inline error appears with a shortcut to view the existing report instead of creating a duplicate.
- A 3-step visual indicator (Select → Parse → Explore) shows progress through the upload.

Recent Reports panel (right-hand side)

A scrollable list of previously uploaded reports, each showing its date, "time ago," test count, a small pass-rate badge and a decorative sparkline. Clicking any entry jumps to that report's full Analysis page. This panel has its own Environment and Report Type dropdowns so it can be browsed independently of what's selected elsewhere.

4.2 Trends (Home Page)

/ and /trends

The landing page and the most feature-rich screen in the dashboard — a full overview of quality trends across every uploaded report, filterable by Environment, Report Type, and a custom date range.

Filter bar

Report Type selector, plus a **Date Range** filter (from/to date pickers with Search and Clear) — changes are staged and only applied when Search is clicked.

Summary cards

Avg Pass Rate **Avg Fail Rate** — each averaged over the last 3 uploaded reports in the current filter, with a colored delta showing how the very latest run compares to the one before it.

Charts (last 8 runs, shown once 2+ reports are available)

Chart	What it shows
Test Results by Date	Stacked bar chart of pass/fail/skip/flaky counts per report
Suite Duration Trend	Run time per report, with a Total/Per-Test toggle and colour-banded Fast/Moderate/Slow bars against a dashed average line
Pass Trend	Line chart of pass rate % over time for the selected pipeline
Fail Trend	Line chart of fail rate % over time for the selected pipeline

Latest Runs cards

Up to three cards — **Estimation-API tests** **Estimation AI-API tests** **Estimation AI-UI tests** — each showing the most recent run for that pipeline as a small pass-rate donut plus a 5-tile KPI row (Total, Passed, Failed, Flaky, Skipped) with deltas versus the previous run of that same pipeline. Which cards appear follows the Report Type filter above them.

Test Results Heatmap ("Failures by Folder")

An interactive folder-tree grid — coloured tiles per top-level test folder (green = no failures, shades of red = higher failure rate). Click a tile to descend into subfolders, and clicking a leaf folder opens a panel listing its currently-failing tests with expandable error details. Has its own independent date picker and an Expand/Collapse-all control.

Top Failing Scenarios table

The 8 most frequently failing scenarios across all matched reports: scenario name, suite/file, fail count, run count, error category, and last-failed date.

All Reports table

Every matched report as a row — name, date, pass-rate badge, full status counts, duration, a trend arrow versus the previous report, and actions to view or delete (with a confirmation prompt).

Paginated 8 rows at a time.

4.3 Failures (page header shows "Analysis")

/failures

Cross-run failure-pattern analysis — this is where recurring and newly-introduced problems surface.

Summary cards (clickable to expand a detail list)

Card	Meaning
Newly Broken Tests	Regressions — tests that passed before and are now failing
Always Failing	Tests that have failed in every run they've appeared in (2+ runs)
Flaky Tests	Tests whose result flips between pass and fail across consecutive runs, shown with a flakiness % score

Newly Broken Tests — Regression Detector

Compares two dates' worth of reports to find tests that passed on Day 1 but failed on Day 2. Defaults to the two most recent upload dates, but both dates can be changed manually to compare any two points in time. Each regression is shown as a card with the test name, file, error category, error message, expandable stack trace, and a "Passed → Failed" date pill. Exportable as a detailed **CSV** or a minimal **Excel** file (test name + pass/fail per date).

Suite Health

A horizontal bar chart of the top 10 test files/suites by failure rate, colour-coded amber to red, with fail rate %, failed count and total runs on hover.

Failure Types by Run

A stacked bar chart of error category counts (Assertion, Timeout, Network, Element, Runtime, Application) per report/date — useful for spotting whether failures are shifting from, say, timeouts to assertion errors over time.

Shows a clean "All tests passing" state when there is nothing to report.

4.4 Scenarios

[/api-scenarios](#)

Every individual test scenario, grouped by its API/test file, with a full run-history view across all uploads.

Uploads strip

A row of chips — one per uploaded report, oldest to newest — each showing its date and pass rate %. Hovering a chip highlights that report's column across every scenario's run-history dots below.

Filter bar

Free-text search, a date dropdown (one upload date), a suite dropdown (one API/file), a status dropdown (All / Failed / Flaky / Passed / Skipped, with live counts), and an Expand/Collapse-all control.

API groups

Collapsible sections, one per API/test file, showing failed/flaky badges, total scenario count and a mini pass-rate bar. Expanding a group lists every scenario with a row of small coloured dots — one per historical upload — forming a compact "run history" per scenario, alongside an error category badge and expandable error details.

4.5 Tenant Comparison

/tenant-comparison

Compares how the *same* API test scenarios behave across different customer tenants within one selected report (this page only applies to API-classified reports — UI reports are excluded). A report picker in the header selects which single report to analyse, defaulting to the most recent.

If fewer than two tenants can be detected in a report, the page automatically falls back to comparing the same test across different uploaded runs instead, with a clear label explaining the fallback.

Summary cards (clickable filters)

Card	Meaning
Tenants	Number of distinct tenants detected in this report
Divergent APIs	APIs that pass for some tenants but fail for others — the key signal this page is built to surface
Failing All	Scenarios that fail across every tenant
All Passing	Scenarios passing for every tenant

WHY THIS MATTERS

A divergence banner explains the insight directly, e.g.: "3 APIs behave differently across tenants — likely tenant-specific data, configuration, or permissions issues rather than a code bug." This helps distinguish real bugs from tenant-specific configuration problems.

Layout

A row of small pass-rate donut charts (one per tenant), then tabs — one per tenant plus a **Compare All** tab showing every tenant side by side as a pass/fail matrix. Divergent scenarios are flagged with an amber marker. Filters for All / Has failures / Divergent only, plus search.

4.6 Single-Report Analysis

/analysis/:id — reached by clicking any report elsewhere in the app

The deep-dive view for one specific uploaded report.

- **Executive Summary** banner — an icon and heading reflecting overall health (pass rate $\geq 85\%$ good, $\geq 70\%$ caution, below that a warning).
- **Six stat cards** — Total, Passed, Failed, Skipped, Flaky, Duration. Clicking Passed / Failed / Skipped / Flaky jumps to the Drill-Down page pre-filtered to that status.
- **Test Status Overview** — a donut chart of the same four statuses (also clickable to drill down), paired with a suite-detail panel that fills in when a specific suite is selected from the bar chart alongside it.
- **Results by Suite** — a stacked bar chart (Passed/Failed/Skipped/Flaky) per test suite/file, clickable to populate the suite-detail panel.

4.7 Drill-Down (Test List)

/analysis/:id/category/:status

Reached from the Analysis page's stat cards or donut chart — shows the full list of tests matching one status (Passed / Failed / Skipped / Flaky) for one report.

- For Failed/Flaky lists, a row of **error category pills** (Assertion, Timeout, Element, etc., with counts) lets you filter further.
- Each test is an expandable card: status badge, title, full test ID, duration, retry count, file:line reference, and — if failed — the error category, first line of the error, and a collapsible full stack trace.
- A footer summarises the total matching test count and combined duration for the filtered list.

5. Exports — PDF, CSV & Excel

Every major page has its own **Download PDF** button, producing a tailored export of that page's content (Trends, Failures, Scenarios, Tenant Comparison, single-report Analysis, and Drill-Down all support this independently).

The Failures page additionally offers, specifically for the Newly Broken Tests / regressions list:

- **Download CSV** — full detail export (test name, file, error, category, dates).
- **Excel (API + Pass/Fail)** — a minimal export of just test name plus pass/fail status per compared date, for quick sharing.

6. Access & Security Notes

IMPORTANT — NO LOGIN

There is currently **no authentication or user accounts** on this dashboard. Anyone who can reach the app's URL can view every report and can upload or delete reports — there are no permissions or access levels. This is fine for an internal tool on a trusted network, but worth calling out explicitly to the manager if the dashboard will ever be exposed more broadly, since deleting a report is a destructive, unconfirmed-by-identity action available to any visitor.

Basic hardening that *is* in place: HTTP security headers (via Helmet), CORS restricted to the front-end's own origin, and a rate limit of 2,000 requests per 15 minutes per client.

7. Glossary of Terms

Term	Meaning
Environment	Which test environment a report belongs to — QA, SIT, or PPE. Chosen manually at upload time.
Report Type / Pipeline	Which automated suite a report belongs to: Estimation API, Estimation AI-API, or Estimation AI-UI. Detected automatically from the test data.
Flaky test	A test whose outcome is inconsistent — it passes in some runs and fails in others with no code change in between.
Regression	A test that used to pass and has now started failing — flagged by the "Newly Broken Tests" detector.
Error category	An automatic classification of why a test failed: Assertion, Timeout, Network, Element-not-found, Runtime, or Application.
Tenant	A distinct customer/organisation instance within a multi-tenant product, identified from logged Tenant IDs in test output.
Divergent API	An API that passes for some tenants but fails for others in the same test run — a sign of tenant-specific data or configuration issues rather than a code bug.
Suite	A group of related test cases, typically corresponding to one test file.

Document generated as an internal walkthrough of the current state of the application. Reflects the codebase as of September 9, 2026.